Theory of Algorithms – Algorithm Efficiency Analysis

Patrick Marais
Department of Computer Science
University of Cape Town
patrick@cs.uct.ac.za





Algorithm Efficiency

- We need measures of:
  - Input's size (typically we call this *n*)
  - Unit of measuring time – the basic operation of the algorithm
- We're usually interested in growth order
  - So $O(n^2)$ vs $O(n^3)$ is more important than $1412n^2$ vs $4n^3$
- We are also interested in best, average and worst performance
- Interested in time efficiency (basic ops) as well as memory efficiency (how many extra memory units required
  - Both functions of input size n

Consider Linear Search

- What is its worst case on array of length n?
  - It looks at all n elements. Its worst-case efficiency is Θ(n)
- In the best case it looks at 1 element. Its best-case efficiency is Θ (1)
- On average it will look at n/2 elements before finding the element being searched
  - Its average case efficiency is therefore Θ (n/2)
  - But the ½ is a constant, so it's Θ (n)
- Note: can also use O(1) or O(n), but we want tightest bound

Typical Efficiency Classes

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	n!
10	3.3	10	33.2	1E+02	1E+03	1E+03	4E+06
100	6.6	100	664.4	1E+04	1E+06	1E+30	9E+157
1,000	10.0	1,000	9,965.8	1E+06	1E+09	1E+301	
10,000	13.3	10,000	132,877.1	1E+08	1E+12		
100,000	16.6	100,000	1,660,964.0	1E+10	1E+15		
1,000,000	19.9	1,000,000	19,931,568.6	1E+12	1E+18		

Big O – Intuitively

- Example 1:
  - Algorithm executes: $2n + 10$ operations
  - We are not interested in the 2 or the 10
  - This is $O(n)$
- Example 2:
  - Algorithm executes: $3n^2 + 9n + 5$ operations
  - We are not interested in the 3, 9n or 5
  - This is $O(n^2)$

How Long to Sort 100m Items?

- $n * \log_2 n$ = 2,657,542,476 (in 30s)
- n^2 = 10,000,000,000,000,000
  = 10^{16}
- 10^{16} / 2,657,542,476 = 3,762,875

  3,762,875 x 30 seconds = 112,886,248s
  = 3.6 years
- Insertion sort and Bubblesort are impractical for large data sets!

Definition of Big O

- A function t(n) ∈ O(g(n)) iff there is a c and an n_0 such that t(n) <= cg(n) for all n>=n_0
- Example 1:
  - 100n+5 ∈ O(n)
  - To see that this is the case set c to 101 and n_0 to 5.
  - Then we can verify that 100n+5 <= 101n for all n >= 5.
- Example 2:
  - 100n + 5 ∈ O(n^2). Set c = 21 and n_0 = 5
  - So verify that 100n + 5 <= 21n^2
- More formally:
  - 100n +5 <= 100n +n (n>=5) = 101n <= 101n^2 ∈ O(n^2).

Definition of Big Omega - Ω

- A function t(n) ∈ Ω(g(n)) iff there is a positive constant c and a non-negative integer n_0, such that t(n) >= cg(n) for all n>=n_0
- Same as Big O, except *greater than* instead of *less than* sign
- Not widely used – lower bound on growth

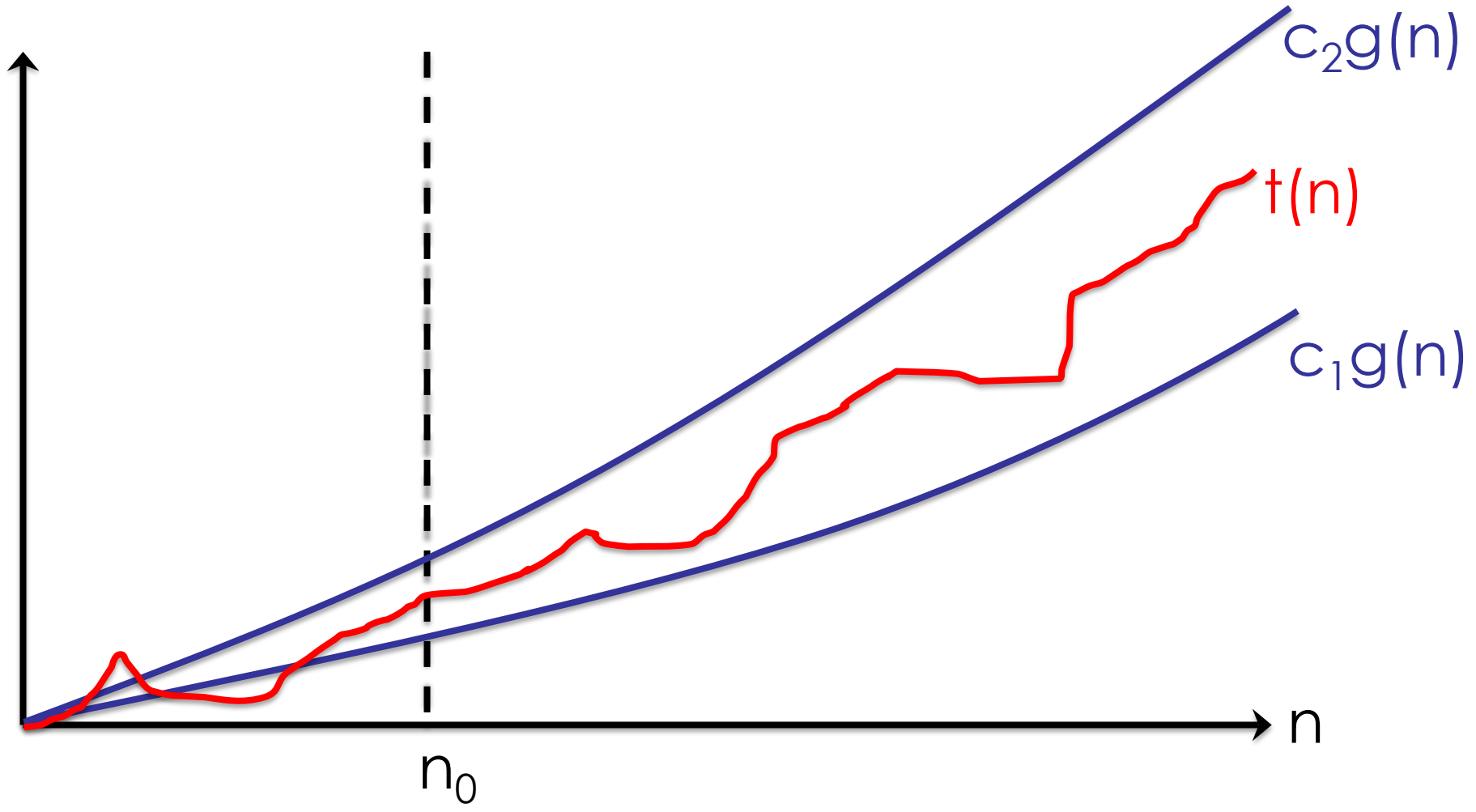
Big Theta - Θ

- Consider sequentially summing an array
  - Its efficiency class is O(n)
  - But it is also by our definition $O(n^2)$ and $O(2^n)$ and O(n log n)
  - NB: It is <u>not</u> O (log n)
- O (n^2) means the algorithm's basic operations execute in proportion to n^2 or better
- How do we say that summing array's basic operation executes EXACTLY proportional to n? We use Big Theta, Θ
  - Efficiency class for summing an array is Θ(n)
  - It is not: $\Theta(n^2)$ or Θ(n log n)
  - It is precisely Θ(n)

Definition of Big Theta - Θ

- A function t(n) ∈ Θ(g(n)) iff there is a c_1, c_2, and n_0 such that $c_2 g(n) <= t(n) <= c_1 g(n)$ for all $n >= n_0$
- Same as big O, except functions in this class cannot be in a more "efficient" class
  - 100n + 5 ∈ Θ(n)
  - 100n + 5 ∈ O(n^2)
  - But 100n + 5 is **NOT** ∈ Θ(n^2)
- We use this a lot

Big Theta (Θ) Graphically



Useful theorem

- If $t1(n) \in O(g(n))$ and $t2(n) \in O(h(n))$ then
  $t1(n) + t2(n) \in O(\max\{g(n), h(n)\})$
- Same holds for Θ and Ω
- This allows us to estimate complexity for algorithms with sequential parts of known complexity
- Example: "bubble sort" an array then find an element
  Sorting: $O(n^2)$
  Search: $O(n)$
  Combined = $O(\max\{n^2, n\}) = O(n^2)$

Comparing Big O, Ω, and Θ

- Big O (big oh)
  - Running time as n gets larger is at most proportional to g(n)
  - Upper bound
- Big Ω (big omega)
  - Running time as n gets larger is at least proportional to g(n)
  - Lower bound
- Big Θ (big theta)
  - Running time as n gets larger is exactly proportional to g(n)
  - Exact bound

Examples

- Average of $O(n^2)$ – algorithm grows at most as fast as n^2 with average case input
  - E.g., Bubble sort
- Worst case of $O(n^3)$ – algorithm grows at most as fast as n^3 with its worst case
  - E.g., brute force matrix multiplication, which is also $\Theta(n^3)$
- Best case of $\Theta(n)$ – algorithm grows linearly in the best case
  - E.g., sum an array
- Worst case of $\Omega(2^n)$ → algorithm grows at best exponentially in worst case
  - E.g., create the power set

Checkpoint

- True or false: $\Theta(n + \log n) = \Theta(n)$
- True or false: $O(n + \log n) = O(n)$
- True or false: $\Theta(n \log_2 n) = \Theta(n \log_{10} n)$
- True or false: $\Theta(\log^2 n) = \Theta(\log n)$
- True or false: $O(n \log n) = O(n)$
- True or false: if $x \in O(n \log n)$ then $x \in O(n^2)$
- True or false: if $x \in \Theta(n \log n)$ then $x \in \Theta(n^2)$
- True or false: if $x \in O(n \log n)$ then $x \in O(n)$

Checkpoint Solutions

- True: $\Theta(n + \log n) = \Theta(n)$
  - We can discard addition terms that grow more slowly
- True: $O(n + \log n) = O(n)$
  - Same as above
- True: $\Theta(n \log_2 n) = \Theta(n \log_{10} n)$
  - $\log_2 n / \log_{10} n$ = a constant = 3.32, approximately (log base change)
- False: $\Theta(\log^2 n) = \Theta(\log n)$
  - $\Theta(\log^2 n)$ grows more than a constant faster than $\Theta(\log n)$
- False: $O(n \log n) = O(n)$
  - These are two different sets. But $O(n)$ is a subset of $O(n \log n)$

Checkpoint Solutions

- True: if $x \in O(n \log n)$ then $x \in O(n^2)$
  - E.g., Mergesort efficiency is in $O(n \log n)$. By definition it is also therefore in $O(n^2)$
- False: if $x \in \Theta(n \log n)$ then $x \in \Theta(n^2)$
  - E.g. Mergesort is strictly $\Theta(n \log n)$
- False: if $x \in O(n \log n)$ then $x \in O(n)$
  - x might be $\in O(n)$ but it might not be. E.g. Mergesort

Show that Θ (n + log n) = Θ (n)

Show that functions in Θ(n + log n) are actually in Θ(n)

- We know that Θ(2n) = Θ(n) (constants don't matter)
- We also know:

  n + log n < n + n (because log n < n)

  => n + log n < 2n

  and we know that: n + log n >= n

So n <= n + log n < 2n

which means t(n) = n+log ∈ Θ (n), thus Θ(n + log n) = Θ(n)

i.e. these two sets of functions are equivalent in terms of asymptotic efficiency

Theory of Algorithms – Efficiency Derivation (Sequential)





Steps for Calculating Efficiency

1. Identify Algorithm's basic operations:
   - The operations that are executed repeatedly at the core of the algorithm
   - E.g., comparisons and swapping in sorting
   - E.g., multiplications and additions in matrix multiplication
2. Set up an equation which counts the number of basic operations for a given input of size n
   - C(n) = ...
3. Solve the equation

Some Simple Maths Tools

- S1 $\sum_{i=j}^{n} 1 = n - j + 1$
- R2 $\sum_{i=1}^{u} ca_i = c\sum_{i=1}^{u} a_i$
- R2 $\sum_{i=1}^{u} a_i \pm b_i = \sum_{i=1}^{u} a_i \pm \sum_{i=1}^{u} b_i$
- S2 $\sum_{i=0}^{n} i = \sum_{i=1}^{n} i = n(n+1)/2 \approx n^2 \in \theta(n^2)$
  - i.e., 1+2+3+…+n = n(n+1)/2

Analysis Exercise 1

```
int MysteryFunction( A [0 .. n-1] )
     MysteryVal = A[0]
     for i = 1 to n - 1:
          if A[i] > MysteryVal:
               MysteryVal = A[i]
     return MysteryVal
```

- What does this algorithm do?
- What is its basic operation?
- In terms of n, how many times is its basic operation executed?
- What is its *asymptotic efficiency class*?

Analysis Solution 1

```
int maxElement( A [0 .. n-1] )
      maxVal = A[0]
      for i = 1 to n - 1:
            if A[i] > maxVal :
                  maxVal = A[i]
      return maxVal
```

- What does this algorithm do?
  - Finds the largest element in an array
- What is its basic operation?
  - `A[i] > maxVal` (comparison)

Analysis Solution 1

```
int maxElement( A [0 .. n-1] )
     maxVal = A[0]
     for i = 1 to n - 1:
          if A[i] > maxVal :
               maxVal = A[i]
     return maxVal
```

- In terms of n, how many times is its basic operation executed?

$$C(n) = \sum_{i=1}^{n-1} 1 = n - 1 \in \theta(n)$$

- What is its *asymptotic efficiency class*? – Θ(n) (also big O!)

Analysis Exercise 2: Is it a set

- You have loaded a file of strings into an array
- You want to check if there are any duplicates
- In other words, is the array, A, a set?
  - i.e., each element appears at most once
- Come up with a few algorithms to determine if A is a set
- What are their time complexities?
- Which is more efficient?

Is it a set – Brute Force

```
boolean isset(A)  {
  for (i=0; i<A.length; i++)
    for (j= 0; j<A.length; j++)
      if (i != j and A[i] == A[j])
          return false;
  return true;
}
```

- What is the worst case?
  - Both loops run to completion
- What are the core operations?
  - comparison
- How many times does the inner loop execute?
  - $(n-1)^2$ comparisons

Is it a set – Decrease & Conquer (iterative)

```
boolean isset(A)  {
  for (i=0; i<A.length-1; i++)
    for (j=i+1; j<A.length; j++)
      if (A[i] == A[j]) return false;
  return true;
}
```

- What is the worst case? – No early out, completion of both nested loops
- What are the core operations? – comparisons
- How many times does the inner loop execute?
  - n-1 + n-2 + n-3 … + 1 times = n(n-1)/2
  - Worst case: $\Theta(n^2)$. Average case too

Full Derivation

$$C_{worst}(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1$$

- S1 on the second summation and simplifying

$$\sum_{i=0}^{n-2} (n-1-i)$$

- Using R2 this simplifies to

$$\sum_{i=0}^{n-2} (n-1) - \sum_{i=0}^{n-2} i$$

- Using R1 on the first summation and S2 on the second

$$(n-1)\sum_{i=0}^{n-2} 1 - \frac{(n-2)(n-1)}{2}$$

- Using S1 on the first summation and simplifying

$$(n-1)^2 - \frac{(n-2)(n-1)}{2} = (n-1)n/2 \approx \frac{1}{2}n^2 \in \theta(n^2)$$

Is it a set – Decrease & Conquer (recursive)

```
Boolean isset(Array A, int index=0) {
  if (index >= A.length-1) return true;
  for (i=index+1; i<A.length; i++) {
    if (A[index]==A[i]) return false;
  }
  return isset(A, index+1);
}
```

- Don't let the recursion scare you
- This algorithm is materially identical to the brute force one
- Worst case: $\Theta(n^2)$. Average case too
- We'll see how to do the maths for this later

Is it a set – Transform & Conquer

```
boolean isset(A) {
   sort(A);
   for (int i=0; i<A.length-1; i++)
     if (A[i] == A[i+1]) return false;
   return true;
}
```

- Here we sort the array and then do a linear search through it
- So which is more efficient? Transform & conquer or brute force?

Brute Force vs. Transform & Conquer

- What is the efficiency of the transform & conquer solution?
  - The most efficient sorting algorithms are $\Theta(n \log n)$
  - The for loop is linear for the worst case: $\Theta(n)$
  - It runs the sort, then the for loop. The asymptotic efficiency of this sequence of 2 computational steps is:

$$\Theta(\max\{n \log n, n\}) = \Theta(n \log n)$$

- So the Transform & Conquer algorithm is faster than the brute force solution of $\Theta(n^2)$

Theory of Algorithms – Efficiency Derivation (Recursive)





Analysing Recursive Algorithms

- Recursive efficiency follows the same pattern as sequential:
  - Determine core operation ⇢ create equation for number of repetitions ⇢ solve ⇢ derive efficiency
- Except that equations are recurrence relations and not summations
  - Need to build a similar maths toolset for recurrence relations
  - Either solve by Backward Substitution or using the Master Theorem (or closed form recurrence relation solutions - ignored, but see Appendices of Levitin)

Recurrence Relations

- A recurrence relation is a recursive mathematical function

$$\text{e.g., } M(n) = \begin{cases} 0 & if\ n = 0 \\ M(n-1) + 1 & otherwise \end{cases}$$

  - Has a recursive and base case
- A tool for analysing recursive algorithms
- We will consider these recurrence relations:
  - T(n) = aT(n-k) + f(n) // Like M(n) above

    OR
  - T(n) = aT(n/b) + f(n)

Analysis Exercise 3: Factorial

```
int F(n) {
      if n == 0 return 1;
      return F(n-1) * n;
}
```

- Basic operation we want to count?
  - Multiplication
- How many multiplications?
  - Once for every recursive call

Analysis Solution 3 by Backward Substitution

- Let M(n) be number of multiplications
- Then $M(0) = 0$

  And $M(n) = M(n-1) + 1$

  $M(n) = M(n-1) + 1 = [M(n-2) + 1] + 1$

  $= [M(n-3) + 1] + 2 = M(n-3) + 3$
- And in general:

  $M(n) = M(n-k) + k$, for k substitutions

  Thus, when $k = n$ (n substitutions)

  $M(n) = M(n-n) + n = M(0) + n = n$

  So $M(n) \in \Theta(n)$ [it is exactly n in this case]

Analysis Exercise 4: Counting Bits

```
int CountBits(int n such that n > 0):
        if n == 1 return 1
        else return 1 + CountBits(n / 2)
```

- How do we determine the number of basic operations?
  - Basic op is addition of 1 on each call to CountBits.

```
int CountBits(int n such that n > 0):
        if n == 1 return 1
        else return 1 + CountBits(n / 2)
```

Analysis Solution 4

- A(1) = 0 The addition doesn't take place when n=1
  A(n) = A(n / 2) + 1 for n > 1
  Actually need $A(\lfloor \frac{n}{2} \rfloor)$ to get integer argument → problem!
- BUT we can use a trick – assume n is a power of 2:
  - Let $n = 2^k$ which is the same as saying $k = \log_2 n$
  - Now $n/2 = (½) \cdot 2^k = 2^{-1}.2^k = 2^{k-1}$
- $A(1) = A(2^0) = 0$
  $A(n) = A(2^k) = A(2^{k-1}) + 1$ for $k > 0$
  $= [A(2^{k-2}) + 1] + 1 = A(2^{k-2}) + 2$
  $= A(2^{k-k}) + k = A(2^0) + k = k = \log_2 n \in \Theta(\log n)$

Extends to all positive n via 'smoothness theorem'

A more complicated case by Backward Substitution

- Let $M(n) = 2\,M(n/2) + 1,\ M(1) = 0$
- Then, we can let $n = 2^k$, and proceed:

  $M(n) = M(2^k) = 2\,M(2^k/2) + 1$

  $\Rightarrow\quad M(2^k) = 2\,\mathbf{M(2^{k-1})} + 1$ NOW recursively expand this….

  $= 2\,(2M(2^{k-1-1}) + 1) + 1$

  $= 2^2\,\mathbf{M(2^{k-2})} + 2 + 1$

  $= 2^2\,(2M(2^{k-2-1}) + 1) + 2 + 1$

  $= 2^3\,M(2^{k-3}) + 2^2 + 2 + 1$
- And in general, after i substitutions:

  $M(2^k) = 2^i\,M(2^{k-i}) + 2^{i-1} + 2^{i-2} + \ldots + 2 + 1$

  When i=k, we get [recall $M(1) = 0$…our base case]

  $= 2^k\,M(1) + 2^i - 1$ [because, for e.g. : $1 + 2 + 4 = 7 = 2^3 - 1$]

So finally, we have $M(2^k) = M(n) = 2^k - 1 = n - 1 \in \Theta(n)$

Fibonacci Analysis

```
int F(n):
        if n<=1 return n
        else return F(n-1) + F(n-2)
```

- Basic operation is an addition
- Recurrence relation:
  - A(n) = A(n-1) + 1 + A(n-2)
  - Requires a different solution method (uses "characteristic equation")
  - $A(n) \in \Theta(1.61803^n)$ (!)

Master Theorem Motivation

- No general solution to all recurrence relations
- But the Master Theorem shows the asymptotic efficiency class (e.g. big theta or big O) of recurrence relations of the form:

$$T(n) = aT(n/b) + f(n) \text{ where } f(n) \in \Theta(n^d)$$

- Provides a short cut that replaces backward substitution
- Proof out of scope for course
  - See Appendix B in Levitin

Master Theorem

- If we have a recurrence of this form:
  - $T(n) = aT(n/b) + f(n)$, $f(n) \in \Theta(n^d)$ and $T(1) = c$ then

    $T(n) \in \Theta(n^d)$ if $a < b^d$

    $T(n) \in \Theta(n^d \log n)$ if $a = b^d$

    $T(n) \in \Theta(n^{\log_b a})$ if $a > b^d$
  - Analogous results for O and Ω
  - You have to know how to apply the Master Theorem
  - You do not have to prove or derive it

Simple Example of Master Theorem

- Consider: $A(n) = A(n / 2) + 1$, $A(1) = 0$
  - For pattern: $T(n) = aT(n/b) + f(n)$, $f(n) \in \Theta(n^d)$ and $T(1) = c$
  - $a = 1$, $b = 2$, $c = 0$, and $f(n) \in \Theta(1) = \Theta(n^0)$
  - Thus, $d = 0$
- Select Master Theorem Case:
  - $b^d = 1$
  - Therefore $a = b^d$
  - By Master Theorem: $T(n) \in \Theta(n^d \log n)$
- And since $d = 0$, $T(n) \in \Theta(\log n)$

Another Simple Example

- Consider: $A(n) = 2\,A(n/2) + 1,\ A(1) = 1$
  - For pattern $T(n) = aT(n/b) + f(n)$, $f(n) \in \Theta(n^d)$ and $T(1) = c$
  - $a = 2,\ b = 2,\ c = 1$
  - and $f(n) \in \Theta(1) = \Theta(n^0)$, so $d = 0$
  - Now $2 > 2^0$ so $a > b^d$
  - By master theorem $A(n) \in \Theta(n^{\log_b a}) = \Theta(n^{\log_2 2}) = \Theta(n)$